

AutoTraQ Testing Checklist — DEAN (Database)

Login: dean@autotraq.app / autotraq2026 **URL:** https://cs490unco.org **API Base:** https://backend-production-b163.up.railway.app **Role:** Manager

Your section is the **database layer**. You'll test data integrity, relationships, persistence, and edge cases. Use both the frontend AND direct API calls to verify data is stored and retrieved correctly.

0. Setup

Log in at https://cs490unco.org with your credentials. Also get an API token:

```
curl -X POST https://backend-production-b163.up.railway.app/api/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"dean@autotraq.app","password":"autotraq2026"}'
```

1. Data Integrity — Parts (12 tasks)

- [] Browse the Parts catalog — verify **534 parts** exist (count shown on dashboard)
- [] Click into 5 different parts — verify each has: name, SKU, description, condition, price
- [] Check that **no parts have NULL/empty names** — scroll through the list looking for blanks
- [] Check that **SKU format is consistent** — note the pattern (e.g., T0Y-CAM-ENG-BLK-U)
- [] Verify **SKUs are unique** — search for any SKU, confirm only 1 result
- [] Check **condition values** — should be consistent (e.g., "New", "Used", "Refurbished")
- [] Find a part and note its **price** — go to Part Detail page — verify the price matches
- [] Check if any parts have **\$0.00 price** — note how many
- [] Check if any parts have **missing descriptions** — note how many
- [] Create a new part via the UI — refresh the page — verify it **persists** (wasn't lost)
- [] Edit that part's name — refresh — verify the **edit persisted**
- [] Delete that part — refresh — verify it's **actually gone**

2. Data Integrity — Vehicles (10 tasks)

- ☐ Browse the Vehicles page — count how many vehicles exist
- ☐ Check that every vehicle has: **Year, Make, Model** (no blanks)
- ☐ Verify **year range makes sense** (should be realistic years like 1990-2026)
- ☐ Check for **duplicate vehicles** — same year/make/model shouldn't appear twice
- ☐ Filter by make "Toyota" — count results, then filter "Ford" — different counts?
- ☐ Create a test vehicle (2024 Honda Civic) — verify it appears in the list
- ☐ Create the **same vehicle again** (2024 Honda Civic) — what happens? Error or duplicate?
- ☐ Edit the vehicle's year to 2025 — verify it saves
- ☐ Delete the test vehicle(s) — verify they're removed
- ☐ Verify the **vehicle count** on the dashboard updates after your changes

3. Data Integrity — Inventory (12 tasks)

- ☐ Check the Inventory page — verify **locations** exist
- ☐ Verify each location has a **name and code**
- ☐ Check **on-hand quantities** — are they reasonable numbers (not negative)?
- ☐ Find a part with inventory > 0 — note the quantity
- ☐ Receive 5 more units of that part — check the quantity increased by exactly 5
- ☐ Make a stock correction (reduce by 2) — verify quantity decreased by exactly 2
- ☐ Check the **inventory events log** — your receive and correction should appear
- ☐ Verify the event log shows: **who** did it, **when**, **what changed**, **how much**
- ☐ Check if **total inventory units** on Dashboard matches sum of all on-hand quantities
- ☐ Look for any parts with **negative stock** — this should NOT happen (flag if it does)
- ☐ Create a new location — receive stock there — verify it shows up
- ☐ Try receiving **0 units** — what happens? (Should be rejected)

4. Data Integrity — Requests (8 tasks)

- ☐ Create a part request (quantity: 3)
- ☐ Verify the request shows status "PENDING"
- ☐ Approve the request — verify status changes to "APPROVED"
- ☐ Fulfill the request — verify status changes to "FULFILLED"
- ☐ Check if fulfillment **affected inventory levels** (stock should decrease)
- ☐ Create another request and **cancel** it — verify status is "CANCELLED"
- ☐ Try creating a request with **quantity 0** — should be rejected
- ☐ Try creating a request for a **non-existent part ID** — should error gracefully

5. Data Integrity — Users & Auth (10 tasks)

- ☐ Go to Admin → Users — verify all 5 team accounts exist
- ☐ Verify **Anson** is the only admin (role: admin)
- ☐ Verify **you** are a manager (role: manager)
- ☐ Check that all users have **unique emails**
- ☐ Check if users have **created timestamps** — are they recent/reasonable?
- ☐ Check if **barcode values** are unique across users
- ☐ Change your password → log out → log in with new password → verify it works
- ☐ Change it back to autotraq2026
- ☐ Try registering a new account with an **existing email** — should be rejected
- ☐ Check the **Audit Log** — verify your login appears as an event

6. Relational Integrity (10 tasks)

- ☐ Find a part → check if it shows which **vehicle(s)** it's compatible with
- ☐ Find a vehicle → check if it shows which **parts** are associated with it
- ☐ Create a request for Part #1 → check the request references the correct part name
- ☐ Check inventory for a part → verify the **part name** displays correctly (not just ID)
- ☐ Check audit logs → verify they reference the correct **user name** (not just user ID)
- ☐ Delete a part that has inventory — what happens? (Should either cascade or prevent deletion)
- ☐ Check if **inventory events** reference the correct part after filtering
- ☐ Create a request → approve → fulfill → check that the **audit trail** shows all 3 steps
- ☐ Verify **notifications** (if any) reference the correct entities
- ☐ Check the **CSV export** — verify part IDs in the CSV match what's in the UI

7. Search & Query Accuracy (8 tasks)

- ☐ Search for "brake" — count results on the UI
- ☐ Search for "BRAKE" (caps) — verify the **same results** appear (case-insensitive)
- ☐ Search for a specific SKU — verify exactly 1 result
- ☐ Filter parts by "New" condition — verify ALL results are actually "New"
- ☐ Filter parts by "Used" condition — verify ALL results are "Used"
- ☐ Sort parts by price ascending — verify the order is actually cheapest to most expensive
- ☐ Sort parts by name A-Z — verify alphabetical order
- ☐ Use Advanced Search hierarchy — drill into a system → component — verify results are actually in that category

8. Persistence & Reliability (8 tasks)

- ☐ Create a part — **close the browser** — reopen — is the part still there?
- ☐ Make 5 changes in quick succession (create, edit, delete) — verify all completed correctly
- ☐ Open the app in **two browser tabs** — make a change in tab 1 — refresh tab 2 — does tab 2 show the update?
- ☐ Create a part with **maximum length fields** (very long name, long description) — does it save?
- ☐ Create a part with **minimum data** (only required fields) — does it save?
- ☐ Create a part with **special characters** in the name (& < > " ' / \ @ #) — does it store and display correctly?
- ☐ Create a part with **unicode characters** (é, ñ,) — does it save?
- ☐ Check the database isn't losing data across page refreshes — dashboard stats should be consistent

9. Audit Trail Verification (6 tasks)

- ☐ Navigate to /audit — verify the log has entries
- ☐ Check that **every action you took** during this testing appears in the audit log
- ☐ Verify each entry has: timestamp, user, action type, entity type, entity ID
- ☐ Filter by entity type "parts" — verify only part-related entries show
- ☐ Check that audit entries are in **chronological order** (newest first or oldest first — note which)
- ☐ Verify **no gaps** — if you created 3 things, all 3 should have audit entries

10. Edge Cases & Stress (8 tasks)

- ☐ Try creating a part with **price = 99999999.99** — does it handle large numbers?
- ☐ Try creating a part with **price = 0.001** — does it handle small decimals?
- ☐ Try submitting a form with **only spaces** in text fields — should be rejected
- ☐ Try creating a vehicle with **year = 1800** — does validation catch it?
- ☐ Try creating a vehicle with **year = 3000** — does validation catch it?
- ☐ Rapidly click "Submit" on a form 5 times — does it create duplicates?
- ☐ Check if the app handles **500+ parts** smoothly — no timeouts on the list page
- ☐ Verify pagination works correctly: page 1 + page 2 items = no overlap, no gaps

Total: 92 tasks

Tips: - Use browser DevTools (F12) → Network tab to see what the API returns - For direct API testing, use **Postman** (free) or **curl** in terminal - Screenshot any bugs you find

When done, compile results and send to Anson.